
ARMATURE

Taking the overhead out of research.

Scientists spend most of their week on work that is not science. This document sets out what that work is, why it has not been fixed, and what Armature removes.

ABSTRACT

Scientists spend most of their week on work that is not science. It grows with every person, dataset and system the organisation adds, and only the scientist absorbs it. Armature reaches data where it already sits and lets scientists build their own automations. The work is done once and then runs.

1. THE PROBLEM

Scientists do not spend most of their time on science. They spend it on the work around it. Finding a file. Pulling a number out of it. Putting it in the shape a report needs. Checking whether someone already ran this a year ago.

HANDOFF

One movement of data by hand, from where it is held to where it is needed. A figure copied into a report. A result sent to a colleague. Each one is small, and the number of them scales with the size of the organisation.

No single one of these jobs is large, and there are hundreds of them every week. They sit on no plan and belong to no role, so the time comes straight out of the research.

The material makes it harder. One question spans a process document, a paper, a spreadsheet and an instrument export. Each was written in the shape that suited whatever produced it, and none of them is in the shape the question needs. Closing that gap is the work.[•]

The problem grows with the work. More people, more data and more sources mean more handoffs, not fewer.

In a regulated lab this shows up as a second cost on top of the first: the record of what happened has to be reconstructed from those same scattered handoffs, after the fact, instead of already existing.

2. WHY IT IS NOT FIXED

Nobody owns it, and nobody can fix it alone. It appears in no job description and no budget line. The person who knows what needs

- Some of the most valuable material is also the hardest to reach. Datasets of tens of terabytes cannot be moved somewhere more convenient, and in many organisations they are not permitted to leave the building.

doing cannot build the fix, and the person who can build it does not know the science well enough to get it right. Every fix is also specific to one lab, one instrument and one file format, so no product generalises across them.

The data will not move. Much of it is too large or too sensitive to leave the building, and most current answers begin by asking the organisation to move it first. That is a programme rather than a product, and the work of finding things is still waiting when it finishes.

Private fixes leave with their owner. A script on one laptop. A folder structure only one person understands. Somebody who remembers which version is current. All of it works. None of it is visible, checked or shared, and it leaves when they do.

3. HOW ARMATURE SOLVES IT

3.1 Reach the data where it is

SOURCE

Any system that holds research material: the drives, mail accounts, workspaces and datasets. Armature records what is inside a source and where each item sits. It does not hold the item.

Armature connects to the systems the organisation already uses. Nothing is migrated and nothing is replaced. A source is connected where it is and stays where it is.

Armature keeps a reference to each item rather than the item itself, so a source can be connected without its contents being copied anywhere. That is what serves the data which cannot move. When Armature returns something, it returns the original passage and a pointer to the exact place it came from.

3.2 The scientist builds the automation

CODE ARTEFACT

The program Armature builds from the scientist's description. It is kept, read and versioned like any other piece of code, making a repeated job inspectable. The program does exactly what it says, every time - no more, no less.

Finding the material is half the job. The other half is doing something with it. The scientist describes that job in plain language, and Armature turns it into a short program that performs it. The scientist reads the program, approves it, and it runs.

The program is the artefact that is kept. A colleague can read it. It performs the same steps every time. Nothing is written back to a connected system without a human approval, which is written to an auditable record; along with the inputs, outputs and timings.

The person with the problem is the person who builds the fix, because the knowledge needed to get it right is theirs.



Figure 1. The same weekly job before and after. Today it is rebuilt by hand each time. With Armature it is described once, approved once, and then runs under the same check every time after.

4. WHAT CHANGES

BOUNDARIES

Armature is not an ELN, a LIMS or a QMS, and does not replace one. It does not host storage, and it cannot widen anyone's access: permissions are inherited from the connected systems and enforced.

Time returns to the research. A job that took an hour last week takes five minutes this week, and then takes none at all once it runs on its own.

Work stops being repeated. The organisation can see what it has already done, so the same screen is not run again a year later by someone who never knew about the first one.

Private fixes become shared assets. The script that lived on one laptop becomes an automation the team can read, check and reuse. It stays when the person leaves.

The record built along the way is the same one an audit asks for later. The story kept as it happens, not reconstructed afterwards.

Capacity rises without headcount. The same people deliver more research, because the overhead no longer scales with the organisation.

5. WHERE IT STARTS

Nothing has to change first. A lab connects the source that holds the work, takes one job that is done every week, and builds the automation for it. That job stops being done by hand, and the next one is easier than the first.